
注意力机制就是你所需要的一切

Attention Is All You Need

Ashish Vaswani · Noam Shazeer · Niki Parmar · Jakob Uszkoreit

Llion Jones · Aidan N. Gomez · Łukasz Kaiser · Illia Polosukhin

NeurIPS 2017 · arXiv:1706.03762v7 (2023-08-02)

译注：本文按原论文结构逐段精译。attention 统一译为“注意力机制”，self-attention 译为“自注意力”，multi-head attention 译为“多头注意力”。公式、符号、数值、数据集名与模型名保持原貌；文内引用编号保留；参考文献部分按用户要求不翻译，完整保留英文原文。原论文插图中的英文标签保留，并在图注后给出必要的中文术语对照。

原文许可说明

在适当注明来源的前提下，Google 许可仅为新闻或学术作品之用途复制本文中的表格与图形。

摘要

当前占主导地位的序列转换模型，建立在包含编码器与解码器的复杂循环神经网络或卷积神经网络之上。表现最好的模型还会通过注意力机制连接编码器和解码器。我们提出一种新的、简洁的网络架构——Transformer。它完全以注意力机制为基础，彻底舍弃循环与卷积。两项机器翻译任务上的实验表明：该模型不仅质量更高，而且并行化程度更强，所需训练时间也显著更少。在 WMT 2014 英语到德语翻译任务上，我们的模型取得 28.4 BLEU，较此前包括集成模型在内的最佳结果提高了 2 个以上 BLEU；在 WMT 2014 英语到法语翻译任务上，我们的单模型在 8 块 GPU 上训练 3.5 天后取得 41.8 BLEU，刷新了当时的单模型最佳成绩，而训练成本仅为已有最佳模型的一小部分。通过将 Transformer 成功用于训练数据充足和受限两种情形下的英语成分句法分析，我们还表明了它能够良好地泛化到其他任务。

作者贡献说明

所有标注星号的作者贡献相同，署名顺序随机。Jakob 提出用自注意力替代 RNN，并启动了对这一想法的评估；Ashish 与 Illia 设计并实现了最初的 Transformer 模型，且深度参与了本工作的各个方面；Noam 提出缩放点注意力、多头注意力以及无参数的位置表示，并成为另一位几乎参与全部细节的作者；Niki 在最初代码库与 tensor2tensor 中设计、实现、调优和评估了大量模型变体；Llion 也试验了新的模型变体，负责最初代码库、高效推理与可视化；Lukasz 与 Aidan 用许多个漫长的工作日设计并实现 tensor2tensor 的多个部分，以其替换早期代码库，从而显著提升结果并大幅加速研究。Aidan 的工作完成于 Google Brain 任职期间；Illia 的工作完成于 Google Research 任职期间。

1 引言

循环神经网络——尤其是长短期记忆网络 (LSTM) [13] 与门控循环神经网络 [7]——已经被牢固确立为序列建模和序列转换问题中的先进方法，例如语言建模与机器翻译 [35, 2, 5]。此后，大量研究继续推进循环语言模型以及编码器-解码器架构的性能边界 [38, 24, 15]。

循环模型通常沿输入与输出序列中的符号位置分解计算。它们把序列位置与计算时间步对齐，并令每一时刻的隐藏状态 h_t 由前一隐藏状态 h_{t-1} 与位置 t 的输入共同决定。该机制固有的顺序性，使单个训练样本内部无法并行计算；当序列较长时，这一问题尤为关键，因为内存限制还会降低跨样本批处理的能力。近期工作借助因式分解技巧 [21] 与条件计算 [32] 显著提升了计算效率，后者还改善了模型性能。然而，顺序计算这一根本约束依然存在。

注意力机制已经成为许多优秀序列建模与序列转换模型不可或缺的组成部分。它使模型能够建立依赖关系，而不受相关元素在输入或输出序列中相隔距离的影响 [2, 19]。不过，除少数工作 [27] 外，此前的注意力机制几乎总是与循环网络配合使用。

本文提出 Transformer：一种完全摒弃循环结构、仅依靠注意力机制来捕获输入与输出之间全局依赖关系的模型架构。Transformer 能够实现显著更强的并行化；在 8 块 P100 GPU 上仅训练 12 小时，便可在翻译质量上达到新的先进水平。

2 背景

减少顺序计算同样是 Extended Neural GPU [16]、ByteNet [18] 和 ConvS2S [9] 的出发点。这些模型都以卷积神经网络为基本构件，为所有输入与输出位置并行计算隐藏表示。在这些模型中，为了让任意两个输入或输出位置的信号彼此关联，所需操作数会随位置间距离增长：ConvS2S 中为线性增长，ByteNet 中为对数增长。这使得远距离依赖关系更难学习 [12]。在 Transformer 中，任意位置之间只需常数级操作即可关联；代价是对经注意力加权的位置取平均会降低有效分辨率。我们使用第 3.2 节所述的多头注意力来抵消这一影响。

自注意力 (self-attention，有时称为序列内注意力) 通过关联同一序列中的不同位置来计算该序列的表示。它已成功用于阅读理解、抽象式摘要、文本蕴含以及学习与具体任务无关的句子表示等多种任务 [4, 27, 28, 22]。

端到端记忆网络采用循环式注意力机制，而不是与序列位置对齐的循环结构；它在简单语言的问答和语言建模任务上表现良好 [34]。

据我们所知，Transformer 是第一个完全依靠自注意力计算输入与输出表示、而不使用与序列对齐的 RNN 或卷积的序列转换模型。下文将介绍 Transformer，说明采用自注意力的动机，并讨论它相较于 [17, 18] 和 [9] 等模型的优势。

3 模型架构

多数具有竞争力的神经序列转换模型都采用编码器-解码器结构 [5, 2, 35]。编码器把由符号表示构成的输入序列 $(x_0, \dots, x_0)$ 映射为连续表示序列 $z = (z_0, \dots, z_0)$ 。给定 z ，解码器每次生成一个符号，逐步产生输出序列 $(y_0, \dots, y_0)$ 。模型在每一步都以自回归方式工作 [10]：生成下一个符号时，把此前已经生成的符号作为额外输入。

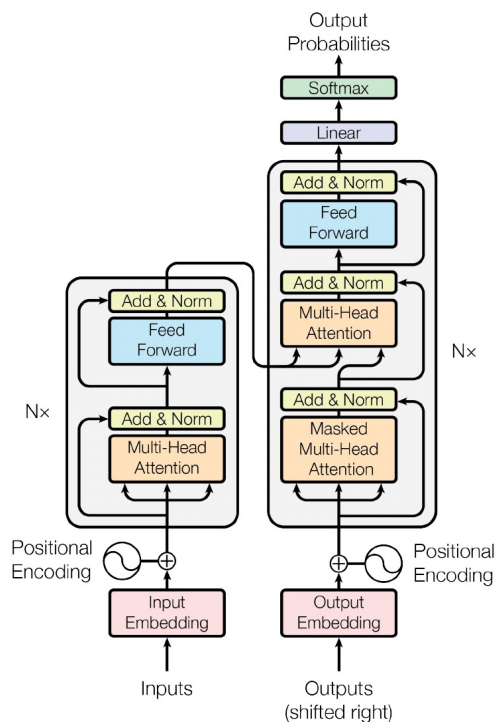


图 1: Transformer 模型架构。图内术语对照: Inputs = 输入; Outputs (shifted right) = 右移后的输出; Input/Output Embedding = 输入/输出嵌入; Positional Encoding = 位置编码; Multi-Head Attention = 多头注意力; Masked Multi-Head Attention = 掩码多头注意力; Feed Forward = 前馈网络; Add & Norm = 残差连接与层归一化; Linear = 线性层; Softmax = Softmax; Output Probabilities = 输出概率。

Transformer 遵循上述总体结构。编码器和解码器都由堆叠的自注意力层与逐位置全连接层构成，分别对应图 1 的左半部分与右半部分。

3.1 编码器与解码器堆栈

编码器：编码器由 $N = 6$ 个相同层堆叠而成。每一层包含两个子层：第一个是多头自注意力机制，第二个是简单的逐位置全连接前馈网络。我们在每个子层外使用残差连接 [11]，随后进行层归一化 [1]。也就是说，每个子层的输出为 $\text{LayerNorm}(x + \text{Sublayer}(x))$ ，其中 $\text{Sublayer}(x)$ 表示该子层本身实现的函数。为了便于使用残差连接，模型中的所有子层以及嵌入层都输出 $d_{\text{model}} = 512$ 维表示。

解码器：解码器同样由 $N = 6$ 个相同层堆叠而成。除编码器每层已有的两个子层外，解码器还插入第三个子层，对编码器堆栈的输出执行多头注意力。与编码器类似，我们在每个子层外使用残差连接，随后进行层归一化。我们还修改变码器堆栈中的自注意力子层，防止某一位置关注其后的未来位置。该掩码与右移一位的输出嵌入共同保证：位置 i 的预测只能依赖位置小于 i 的已知输出。

3.2 注意力机制

注意力函数可以描述为：把一个查询（query）以及一组键-值（key-value）对映射为一个输出。查询、键、值和输出都是向量。输出是所有值的加权和；分配给每个值的权重，由查询与对应键之间的相容性函数计算得到。

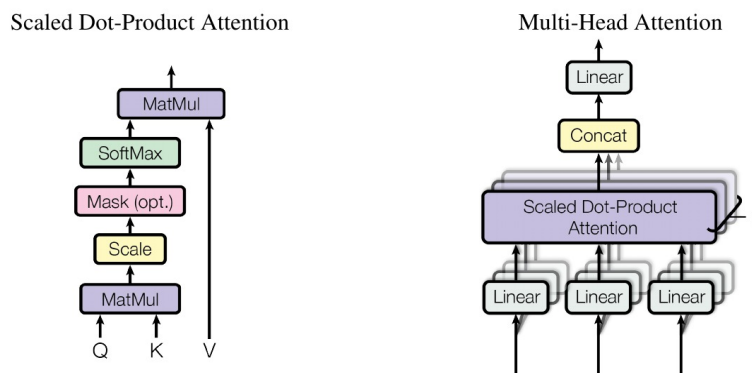


图 2：（左）缩放点积注意力；（右）多头注意力由若干个并行运行的注意力层构成。图内术语对照：MatMul = 矩阵乘法；Scale = 缩放；Mask (opt.) = 可选掩码；Concat = 拼接；Linear = 线性投影。

3.2.1 缩放点积注意力

我们把所采用的注意力称为“缩放点积注意力”（图 2）。其输入由查询、键和值构成：查询和键的维度为 d_Q ，值的维度为 d_V 。我们计算查询与所有键的点积，把每个点积除以 $\sqrt{d_Q}$ ，再应用 softmax 函数得到各个值的权重。

在实际实现中，我们会同时对一组查询计算注意力，并把它们合并为矩阵 Q；键和值也分别合并为矩阵 K 与 V。输出矩阵的计算方式为：

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_Q}) V \quad (1)$$

最常用的两类注意力函数是加性注意力 [2] 与点积（乘性）注意力。除缩放因子 $1/\sqrt{d_Q}$ 外，点积注意力与我们的方法完全相同。加性注意力则使用具有一个隐藏层的前馈网络计算相容性函数。二者的理论复杂度相近，但点积注意力在实践中更快、空间效率更高，因为它可以使用高度优化的矩阵乘法代码实现。

当 d_Q 较小时，两种机制的表现相近；但当 d_Q 较大时，加性注意力优于未经缩放的点积注意力 [3]。我们推测， d_Q 较大时点积的绝对值会变大，把 softmax 推入梯度极小的区域。为抵消这一影响，我们用 $1/\sqrt{d_Q}$ 对点积进行缩放。

脚注：为说明点积为何会变大，设 q 与 k 的各分量是均值为 0、方差为 1 的相互独立随机变量，则其点积 $q \cdot k = \sum_{i=1}^n q_i k_i$ 的均值为 0、方差为 d_Q 。

3.2.2 多头注意力

与其对 d_{model} 维的键、值和查询只执行一次注意力函数，我们发现，把查询、键和值分别以 h 组不同且可学习的线性投影映射到 d_Q 、 d_K 和 d_V 维，会带来更好的效果。随后，我们对这些投影后的查询、键和值并行执行注意力函数，每个头产生 d_V 维输出。最后把各个输出拼接，并再次投影，得到图 2 所示的最终结果。

多头注意力使模型能够在不同位置，联合关注来自不同表示子空间的信息。若只有一个注意力头，取平均会抑制这种能力。

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_O$$

$$\text{head}_i = \text{Attention}(QW_{iQ}, KW_{iK}, VW_{iV})$$

其中投影参数矩阵 $W_{iQ} \in \mathbb{R}^{(d_{\text{model}} \times d_i)}$ 、 $W_{iK} \in \mathbb{R}^{(d_{\text{model}} \times d_i)}$ 、 $W_{iV} \in \mathbb{R}^{(d_{\text{model}} \times d_i)}$ ，而 $W_O \in \mathbb{R}^{(h \cdot d_i \times d_{\text{model}})}$ 。本文使用 $h = 8$ 个并行注意力层（即 8 个头），每个头取 $d_i = d_o = d_{\text{model}}/h = 64$ 。由于每个头的维度降低，总计算成本与具有完整维度的单头注意力相近。

3.2.3 注意力在本模型中的三种用法

Transformer 以三种不同方式使用多头注意力：

- 在“编码器-解码器注意力”层中，查询来自前一个解码器层，作为记忆的键和值来自编码器的输出。这样，解码器的每个位置都能关注输入序列中的所有位置。这模拟了 [38, 2, 9] 等序列到序列模型中典型的编码器-解码器注意力机制。
- 编码器包含自注意力层。在自注意力层中，所有键、值和查询均来自同一处；这里就是编码器前一层的输出。因此，编码器中的每个位置都能关注前一层的所有位置。
- 同样，解码器中的自注意力层允许每个位置关注解码器中截至该位置的所有位置。为保持自回归性质，必须阻止未来信息向当前位置流动。我们在缩放点积注意力内部实现这一点：把 softmax 输入中所有对应非法连接的值屏蔽为 $-\infty$ 。见图 2。

3.3 逐位置前馈网络

除注意力子层外，编码器和解码器的每一层都包含一个全连接前馈网络；该网络分别、且以相同方式应用到每个位置。它由两个线性变换组成，中间使用 ReLU 激活。

$$\text{FFN}(x) = \max(0, xW_1 + b_1) W_2 + b_2 \quad (2)$$

线性变换在同一层的不同位置之间共享，但不同层使用不同参数。也可以把它描述为两层卷积核大小为 1 的卷积。输入和输出维度均为 $d_{\text{model}} = 512$ ，内层维度为 $d_{\text{ff}} = 2048$ 。

3.4 嵌入与 Softmax

与其他序列转换模型类似，我们使用学习得到的嵌入，把输入词元和输出词元转换为 d_{model} 维向量。我们还使用常规的可学习线性变换与 softmax 函数，把解码器输出转换为下一个词元的预测概率。与 [30] 类似，本模型在两个嵌入层和 softmax 前的线性变换之间共享同一个权重矩阵。在嵌入层中，我们把这些权重乘以 $\sqrt{d_{\text{model}}}$ 。

3.5 位置编码

由于模型既不包含循环，也不包含卷积，为了使它利用序列顺序，必须向模型注入词元在序列中的相对或绝对位置信息。为此，我们在编码器和解码器堆栈底部，把“位置编码”加入输入嵌入。位置编码与嵌入具有相同维度 d_{model} ，因此二者能够直接相加。位置编码有许多可选形式，包括学习式与固定式 [9]。

本文采用不同频率的正弦与余弦函数：

$$\text{PE}(\text{pos}, 2i) = \sin(\text{pos} / 10000^{(2i/d_{\text{model}})})$$

$$\text{PE}(\text{pos}, 2i+1) = \cos(\text{pos} / 10000^{(2i/d_{\text{model}})})$$

其中 pos 是位置, i 是维度。也就是说, 位置编码的每个维度对应一条正弦曲线, 其波长以几何级数从 2π 增长到 $10000 \cdot 2\pi$ 。我们选择这一函数, 是因为我们推测它能让模型轻松学会按相对位置施加注意力: 对于任意固定偏移 k , $\text{PE}_{\text{pos}+k}$ 都可以表示为 PE_{pos} 的线性函数。

我们也试验了学习得到的位置嵌入 [9], 发现两种版本得到的结果几乎完全相同 (见表 3 的 (E) 行)。最终选择正弦版本, 是因为它可能使模型外推到比训练时所见序列更长的序列。

4 为什么选择自注意力

本节比较自注意力层与常用于以下映射的循环层、卷积层: 把可变长度的符号表示序列 $(x_0, \dots, x_\ell)$ 映射为等长序列 $(z_0, \dots, z_\ell)$, 其中 $x_i, z_i \in \mathbb{R}^d$ 。这类映射例如典型序列转换编码器或解码器中的隐藏层。为了说明采用自注意力的动机, 我们考虑三个期望性质。

第一, 是每层的总计算复杂度。第二, 是可并行化的计算量, 这里用所需顺序操作的最少次数衡量。

第三, 是网络中长距离依赖关系之间的路径长度。学习长距离依赖是许多序列转换任务的核心难题。影响这种学习能力的关键因素之一, 是前向与反向信号在网络中必须穿过的路径长度。输入与输出序列中任意位置组合之间的路径越短, 就越容易学习长距离依赖 [12]。因此, 我们还比较了由不同类型层构成的网络中, 任意两个输入、输出位置之间的最大路径长度。

表 1: 不同层类型的最大路径长度、每层复杂度与最少顺序操作数。n 为序列长度, d 为表示维度, k 为卷积核大小, r 为受限自注意力的邻域大小。

层类型	每层复杂度	顺序操作数	最大路径长度
自注意力	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
循环层	$O(n \cdot d^2)$	$O(n)$	$O(n)$
卷积层	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log \ell(n))$
受限自注意力	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

如表 1 所示, 自注意力层只需常数级顺序执行的操作便能连接所有位置, 而循环层需要 $O(n)$ 次顺序操作。从计算复杂度看, 当序列长度 n 小于表示维度 d 时, 自注意力层比循环层更快; 采用词片 [38] 与字节对 [31] 表示的先进机器翻译模型, 其句子表示通常满足这一条件。对于很长序列的任务, 可以把自注意力限制为: 每个输出位置只考察输入序列中以该位置为中心、大小为 r 的邻域, 以改善计算性能。这会使最大路径长度增至 $O(n/r)$ 。我们计划在未来进一步研究这一方法。

当卷积核宽度 $k < n$ 时, 单个卷积层无法连接所有输入与输出位置对。若采用连续卷积核, 需要堆叠 $O(n/k)$ 个卷积层; 若采用空洞卷积 [18], 则需 $O(\log \ell(n))$ 层。这会增加网络中任意两位置间的最长路径。卷积层的开销通常比循环层高 k 倍。不过, 深度可分离卷积 [6] 可把复杂度显著降至 $O(k \cdot n \cdot d + n \cdot d^2)$ 。即使 $k = n$, 可分离卷积的复杂度仍等于一个自注意力层与一个逐位置前馈层的总和——也就是本模型采用的组合。

自注意力还可能带来一个附带收益: 模型更易解释。我们检查了模型的注意力分布, 并在附录中展示、讨论若干例子。单个注意力头不仅明显学会了不同任务, 许多头还呈现出与句法和语义结构相关的行为。

5 训练

本节说明模型的训练方案。

5.1 训练数据与批处理

我们在标准 WMT 2014 英语-德语数据集上训练；该数据集约含 450 万个句子对。句子使用字节对编码 [3]，源语言与目标语言共享约 37,000 个词元的词表。对于英语-法语任务，我们使用规模大得多的 WMT 2014 英语-法语数据集，共 3,600 万个句子，并把词元切分为含 32,000 个词片的词表 [38]。句子对按近似序列长度组成批次。每个训练批次所含句子对合计约有 25,000 个源词元与 25,000 个目标词元。

5.2 硬件与训练计划

我们在一台配备 8 块 NVIDIA P100 GPU 的机器上训练模型。对全文采用相应超参数的基础模型而言，每个训练步约耗时 0.4 秒；基础模型共训练 100,000 步，即 12 小时。对大型模型（见表 3 最后一行），每步耗时 1.0 秒，共训练 300,000 步，即 3.5 天。

5.3 优化器

我们使用 Adam 优化器 [20]，其中 $\beta_1 = 0.9$ 、 $\beta_2 = 0.98$ 、 $\epsilon = 10^{-9}$ 。训练过程中按以下公式改变学习率：

$$\text{lrate} = d_{\text{model}}^{(-0.5)} \cdot \min(\text{step_num}^{(-0.5)}, \text{step_num} \cdot \text{warmup_steps}^{(-1.5)}) \quad (3)$$

这意味着：在最初 warmup_steps 个训练步中，学习率线性增加；此后按步数平方根的倒数成比例下降。我们取 warmup_steps = 4000。

5.4 正则化

训练期间，我们采用三种正则化方式：

残差 Dropout。在每个子层的输出与子层输入相加并归一化之前，我们对该输出应用 dropout [33]。此外，在编码器和解码器堆栈中，我们也对嵌入与位置编码之和应用 dropout。基础模型使用 $P_{\text{drop}} = 0.1$ 。

标签平滑。训练时，我们使用 $\epsilon_{\text{ls}} = 0.1$ 的标签平滑 [36]。它会使困惑度变差，因为模型学会对预测保持更多不确定性；但它能提高准确率和 BLEU 分数。

6 结果

6.1 机器翻译

表 2：在 WMT newstest2014 英语-德语与英语-法语测试上，Transformer 以显著更低的训练成本取得了优于既有先进模型的 BLEU 分数。

模型	BLEU EN-DE	BLEU EN-FR	训练成本 FLOPs EN-DE	训练成本 FLOPs EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{10}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{10}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{10}$	$1.2 \cdot 10^{21}$
Transformer (基础)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (大型)	28.4	41.8	$2.3 \cdot 10^{10}$	

在 WMT 2014 英语到德语翻译任务上，大型 Transformer（表 2 中的 Transformer (big)）比此前报告的最佳模型（包括集成模型）高出 2.0 个以上 BLEU，取得新的最佳分数 28.4。该模型的配置列于表 3 最后一行，在 8 块 P100 GPU 上训练 3.5 天。即便是基础模型，也以远低于任何竞争模型的训练成本，超过了此前发布的所有模型和集成模型。

在 WMT 2014 英语到法语翻译任务上，我们的大型模型取得 41.0 BLEU；其训练成本不到此前最佳模型的四分之一，却超过了所有已发表的单模型。用于英语到法语的大型 Transformer 使用 $P_{\text{drop}} = 0.1$ ，而不是 0.3。

基础模型采用单个模型，其参数由最后 5 个检查点求平均得到，检查点每隔 10 分钟保存一次；大型模型则对最后 20 个检查点求平均。我们采用束宽 4、长度惩罚 $\alpha = 0.6$ 的束搜索 [38]，这些超参数通过在开发集上的实验选定。推理时将最大输出长度设为输入长度 + 50，并在可能时提前终止 [38]。

表 2 汇总了实验结果，并将翻译质量和训练成本与文献中的其他模型架构进行比较。我们以训练时间、GPU 数量以及对每块 GPU 持续单精度浮点计算能力的估计值三者相乘，估算训练模型所用的浮点操作数。

脚注：对 K80、K40、M40 和 P100，我们分别采用 2.8、3.7、6.0 和 9.5 TFLOPS 作为持续单精度性能估计。

6.2 模型变体

表 3：Transformer 架构的变体。未列出的值与基础模型相同。所有指标均在英语到德语开发集 newstest2013 上计算；困惑度按字节对编码得到的词片统计，不应与按词统计的困惑度直接比较。为提高中文可读性，本表以“变更项”方式重排原表，但数值与分组保持不变。

组别	变更项	设置	开发集 PPL	开发集 BLEU	参数量 $\times 10^9$
基础	完整基础配置	$N=6, d_{\text{model}}=512, d_{\text{ff}}=2048, h=8, d_k=d_v=64, P_{\text{drop}}=0.1, \epsilon_{\text{ls}}=0.1, 100\text{K 步}$	4.92	25.8	65
A	注意力头数	$h=1, d_k=d_v=512$	5.29	24.9	
A	注意力头数	$h=4, d_k=d_v=128$	5.00	25.5	
A	注意力头数	$h=16, d_k=d_v=32$	4.91	25.8	
A	注意力头数	$h=32, d_k=d_v=16$	5.01	25.4	
B	注意力键维度	$d_k=16$	5.16	25.1	58
B	注意力键维度	$d_k=32$	5.01	25.4	60
C	层数	$N=2$	6.11	23.7	36
C	层数	$N=4$	5.19	25.3	50
C	层数	$N=8$	4.88	25.5	80
C	模型维度	$d_{\text{model}}=256, d_k=d_v=32$	5.75	24.5	28
C	模型维度	$d_{\text{model}}=1024, d_k=d_v=128$	4.66	26.0	168
C	前馈内层维度	$d_{\text{ff}}=1024$	5.12	25.4	53
C	前馈内层维度	$d_{\text{ff}}=4096$	4.75	26.2	90
D	dropout	$P_{\text{drop}}=0.0$	5.77	24.6	
D	dropout	$P_{\text{drop}}=0.2$	4.95	25.5	
D	标签平滑	$\epsilon_{\text{ls}}=0.0$	4.67	25.3	
D	标签平滑	$\epsilon_{\text{ls}}=0.2$	5.47	25.7	
E	位置表示	用学习式位置嵌入替代正弦编码	4.92	25.7	
大型	完整大型配置	$N=6, d_{\text{model}}=1024, d_{\text{ff}}=4096, h=16, P_{\text{drop}}=0.3, 300\text{K 步}$	4.33	26.4	213

为评估 Transformer 各组成部分的重要性，我们以不同方式改变基础模型，并在英语到德语翻译的 newstest2013 开发集上测量性能变化。束搜索设置与上一节相同，但不进行检查点平均。结果见表 3。

表 3 的 (A) 组在保持计算量不变的条件下，改变注意力头数以及注意力键、值的维度，具体方式见第 3.2.2 节。单头注意力比最佳设置低 0.9 BLEU；头数过多也会使质量下降。

在 (B) 组中，减小注意力键的维度 d_k 会损害模型质量。这表明判断相容性并不容易，因而比点积更复杂的相容性函数可能有益。在 (C) 与 (D) 组中，我们进一步观察到：正如预期，更大的模型表现更好，而 dropout 对避免过拟合非常有效。在 (E) 组中，我们用学习式位置嵌入 [9] 替换正弦位置编码，结果与基础模型几乎完全一致。

6.3 英语成分句法分析

为了评估 Transformer 能否泛化到其他任务，我们在英语成分句法分析上进行了实验。该任务具有特定挑战：输出受强结构约束，并且明显长于输入。此外，在小数据场景下，RNN 序列到序列模型尚未达到先进水平 [37]。

我们在 Penn Treebank [25] 的 Wall Street Journal (WSJ) 部分上训练一个 4 层、 $d_{\text{model}} = 1024$ 的 Transformer；该部分约含 4 万个训练句子。我们还进行了半监督训练，使用规模更大、约含 1,700 万个句子的高置信度语料与 BerkeleyParser 语料 [37]。仅使用 WSJ 词表大小为 16K；半监督设置下词表大小为 32K。

我们只进行了少量实验，在第 22 节开发集上选择 dropout（包括第 5.4 节的注意力 dropout 与残差 dropout）、学习率与束宽；其他参数与英语到德语翻译基础模型保持不变。推理时，我们把最大输出长度提高到输入长度 + 300；仅使用 WSJ 和半监督两种设置都采用束宽 21、 $\alpha = 0.3$ 。

表 4：Transformer 能够良好泛化到英语成分句法分析（结果来自 WSJ 第 23 节）。

分析器	训练方式	WSJ 第 23 节 F1
Vinyals & Kaiser et al. (2014) [37]	仅 WSJ, 判别式	88.3
Petrov et al. (2006) [29]	仅 WSJ, 判别式	90.4
Zhu et al. (2013) [40]	仅 WSJ, 判别式	90.4
Dyer et al. (2016) [8]	仅 WSJ, 判别式	91.7
Transformer (4 层)	仅 WSJ, 判别式	91.3
Zhu et al. (2013) [40]	半监督	91.3
Huang & Harper (2009) [14]	半监督	91.3
McClosky et al. (2006) [26]	半监督	92.1
Vinyals & Kaiser et al. (2014) [37]	半监督	92.1
Transformer (4 层)	半监督	92.7
Luong et al. (2015) [23]	多任务	93.0
Dyer et al. (2016) [8]	生成式	93.3

表 4 的结果表明，尽管没有针对该任务进行专门调优，我们的模型仍取得了出人意料的良好表现：除循环神经网络语法模型 [8] 外，它优于此前报告的所有模型。

与 RNN 序列到序列模型 [37] 不同，即使只使用 4 万个 WSJ 训练句子，Transformer 仍超过了 BerkeleyParser [29]。

7 结论

本文提出 Transformer：第一个完全基于注意力机制的序列转换模型。它用多头自注意力替换了编码器-解码器架构中最常用的循环层。

在翻译任务上，Transformer 的训练速度显著快于基于循环层或卷积层的架构。在 WMT 2014 英语到德语和英语到法语两项翻译任务上，我们都取得了新的先进结果；在前一任务中，最佳模型甚至超过此前报告的所有集成模型。

我们对基于注意力的模型前景感到振奋，并计划把它们应用到其他任务。未来，我们将把 Transformer 扩展到输入和输出模态不止是文本的问题，并研究局部、受限的注意力机制，以高效处理图像、音频和视频等大规模输入与输出。如何降低生成过程的顺序性，也是我们的研究目标之一。

训练与评估模型所用代码见：<https://github.com/tensorflow/tensor2tensor>。

致谢

感谢 Nal Kalchbrenner 与 Stephan Gouws 提供富有成果的意见、修正与启发。

参考文献（原文保留，不翻译）

- [1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. arXiv preprint arXiv:1607.06450, 2016.
- [2] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. CoRR, abs/1409.0473, 2014.
- [3] Denny Britz, Anna Goldie, Minh-Thang Luong, and Quoc V. Le. Massive exploration of neural machine translation architectures. CoRR, abs/1703.03906, 2017.
- [4] Jianpeng Cheng, Li Dong, and Mirella Lapata. Long short-term memory-networks for machine reading. arXiv preprint arXiv:1601.06733, 2016.
- [5] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. CoRR, abs/1406.1078, 2014.
- [6] Francois Chollet. Xception: Deep learning with depthwise separable convolutions. arXiv preprint arXiv:1610.02357, 2016.
- [7] Junyoung Chung, Çağlar Gülçehre, Kyunghyun Cho, and Yoshua Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. CoRR, abs/1412.3555, 2014.
- [8] Chris Dyer, Adhiguna Kuncoro, Miguel Ballesteros, and Noah A. Smith. Recurrent neural network grammars. In Proc. of NAACL, 2016.
- [9] Jonas Gehring, Michael Auli, David Grangier, Denis Yarats, and Yann N. Dauphin. Convolutional sequence to sequence learning. arXiv preprint arXiv:1705.03122v2, 2017.
- [10] Alex Graves. Generating sequences with recurrent neural networks. arXiv preprint arXiv:1308.0850, 2013.
- [11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pages 770-778, 2016.
- [12] Sepp Hochreiter, Yoshua Bengio, Paolo Frasconi, and Jürgen Schmidhuber. Gradient flow in recurrent nets: the difficulty of learning long-term dependencies, 2001.
- [13] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. Neural computation, 9(8):1735-1780, 1997.
- [14] Zhongqiang Huang and Mary Harper. Self-training PCFG grammars with latent annotations across languages. In Proceedings of the 2009 Conference on Empirical Methods in Natural Language Processing, pages 832-841. ACL, August 2009.
- [15] Rafal Jozefowicz, Oriol Vinyals, Mike Schuster, Noam Shazeer, and Yonghui Wu. Exploring the limits of language modeling. arXiv preprint arXiv:1602.02410, 2016.
- [16] Łukasz Kaiser and Samy Bengio. Can active memory replace attention? In Advances in Neural Information Processing Systems, (NIPS), 2016.
- [17] Łukasz Kaiser and Ilya Sutskever. Neural GPUs learn algorithms. In International Conference on Learning Representations (ICLR), 2016.
- [18] Nal Kalchbrenner, Lasse Espeholt, Karen Simonyan, Aaron van den Oord, Alex Graves, and Koray Kavukcuoglu. Neural machine translation in linear time. arXiv preprint arXiv:1610.10099v2, 2017.
- [19] Yoon Kim, Carl Denton, Luong Hoang, and Alexander M. Rush. Structured attention networks. In International Conference on Learning Representations, 2017.
- [20] Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In ICLR, 2015.
- [21] Oleksii Kuchaiev and Boris Ginsburg. Factorization tricks for LSTM networks. arXiv preprint arXiv:1703.10722, 2017.
- [22] Zhouhan Lin, Minwei Feng, Cicero Nogueira dos Santos, Mo Yu, Bing Xiang, Bowen Zhou, and Yoshua Bengio. A structured self-attentive sentence embedding. arXiv preprint arXiv:1703.03130, 2017.
- [23] Minh-Thang Luong, Quoc V. Le, Ilya Sutskever, Oriol Vinyals, and Lukasz Kaiser. Multi-task sequence to sequence learning. arXiv preprint arXiv:1511.06114, 2015.
- [24] Minh-Thang Luong, Hieu Pham, and Christopher D Manning. Effective approaches to attention-based neural machine translation. arXiv preprint arXiv:1508.04025, 2015.
- [25] Mitchell P Marcus, Mary Ann Marcinkiewicz, and Beatrice Santorini. Building a large annotated corpus of english: The penn treebank. Computational linguistics, 19(2):313-330, 1993.
- [26] David McClosky, Eugene Charniak, and Mark Johnson. Effective self-training for parsing. In Proceedings of the Human Language Technology Conference of the NAACL, Main Conference, pages 152-159. ACL, June 2006.
- [27] Ankur Parikh, Oscar Täckström, Dipanjan Das, and Jakob Uszkoreit. A decomposable attention model. In Empirical Methods in Natural Language Processing, 2016.
- [28] Romain Paulus, Caiming Xiong, and Richard Socher. A deep reinforced model for abstractive summarization. arXiv preprint arXiv:1705.04304, 2017.

- [29] Slav Petrov, Leon Barrett, Romain Thibaux, and Dan Klein. Learning accurate, compact, and interpretable tree annotation. In Proceedings of the 21st International Conference on Computational Linguistics and 44th Annual Meeting of the ACL, pages 433-440. ACL, July 2006.
- [30] Ofir Press and Lior Wolf. Using the output embedding to improve language models. arXiv preprint arXiv:1608.05859, 2016.
- [31] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. arXiv preprint arXiv:1508.07909, 2015.
- [32] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. arXiv preprint arXiv:1701.06538, 2017.
- [33] Nitish Srivastava, Geoffrey E Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929-1958, 2014.
- [34] Sainbayar Sukhbaatar, Arthur Szlam, Jason Weston, and Rob Fergus. End-to-end memory networks. In C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, and R. Garnett, editors, *Advances in Neural Information Processing Systems 28*, pages 2440-2448. Curran Associates, Inc., 2015.
- [35] Ilya Sutskever, Oriol Vinyals, and Quoc VV Le. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems*, pages 3104-3112, 2014.
- [36] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. *CoRR*, abs/1512.00567, 2015.
- [37] Vinyals & Kaiser, Koo, Petrov, Sutskever, and Hinton. Grammar as a foreign language. In *Advances in Neural Information Processing Systems*, 2015.
- [38] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. Google's neural machine translation system: Bridging the gap between human and machine translation. arXiv preprint arXiv:1609.08144, 2016.
- [39] Jie Zhou, Ying Cao, Xuguang Wang, Peng Li, and Wei Xu. Deep recurrent models with fast-forward connections for neural machine translation. *CoRR*, abs/1606.04199, 2016.
- [40] Muhua Zhu, Yue Zhang, Wenliang Chen, Min Zhang, and Jingbo Zhu. Fast and accurate shift-reduce constituent parsing. In *Proceedings of the 51st Annual Meeting of the ACL (Volume 1: Long Papers)*, pages 434-443. ACL, August 2013.

附录：注意力可视化

以下图像保留原论文中的英文词元，以忠实呈现注意力连线 and 模型输入；中文图注解解释其含义。

Attention Visualizations

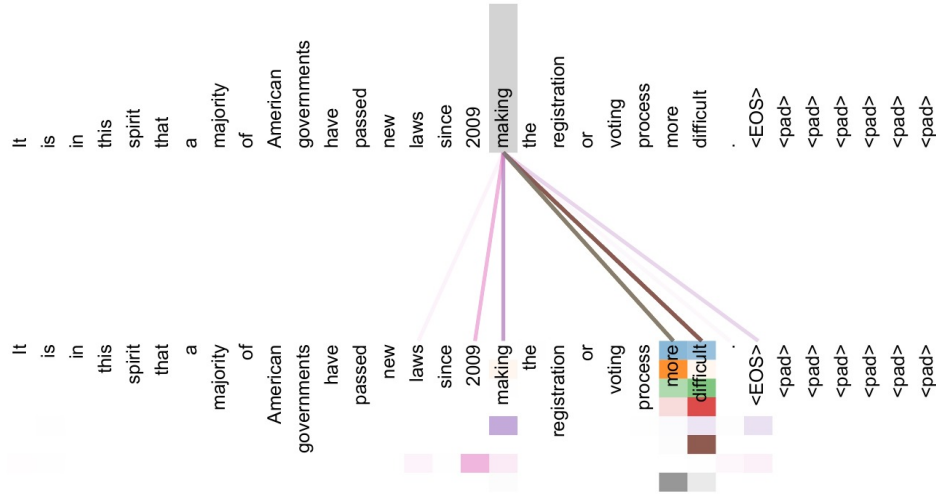


Figure 3: An example of the attention mechanism following long-distance dependencies in the encoder self-attention in layer 5 of 6. Many of the attention heads attend to a distant dependency of the verb 'making', completing the phrase 'making...more difficult'. Attentions here shown only for the word 'making'. Different colors represent different heads. Best viewed in color.

图 3：编码器第 5 层（共 6 层）中的自注意力追踪长距离依赖的示例。许多注意力头都关注动词“making”的远距离依赖，从而补全短语“making ... more difficult”。图中仅显示词语“making”的注意力；不同颜色代表不同注意力头，彩色查看效果最佳。

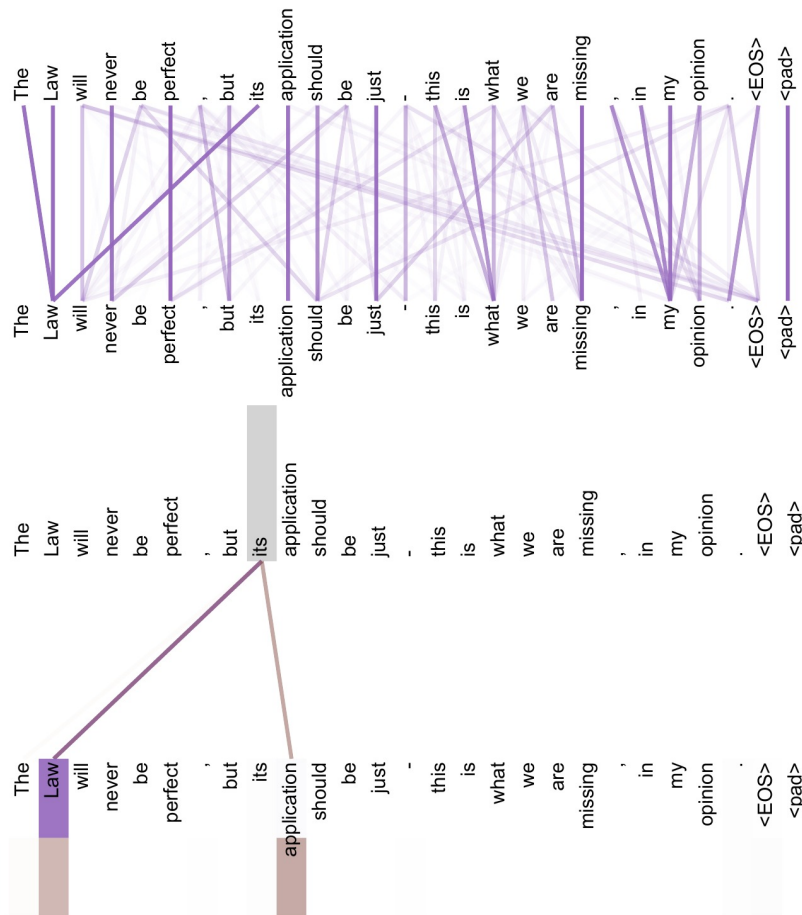


Figure 4: Two attention heads, also in layer 5 of 6, apparently involved in anaphora resolution. Top: Full attentions for head 5. Bottom: Isolated attentions from just the word 'its' for attention heads 5 and 6. Note that the attentions are very sharp for this word.

图 4：同样来自第 5 层（共 6 层）的两个注意力头，它们似乎参与了照应消解。上：第 5 个头的完整注意力；下：仅显示词语 “its” 在第 5、6 个注意力头中的注意力。可见，该词对应的注意力分布十分尖锐。

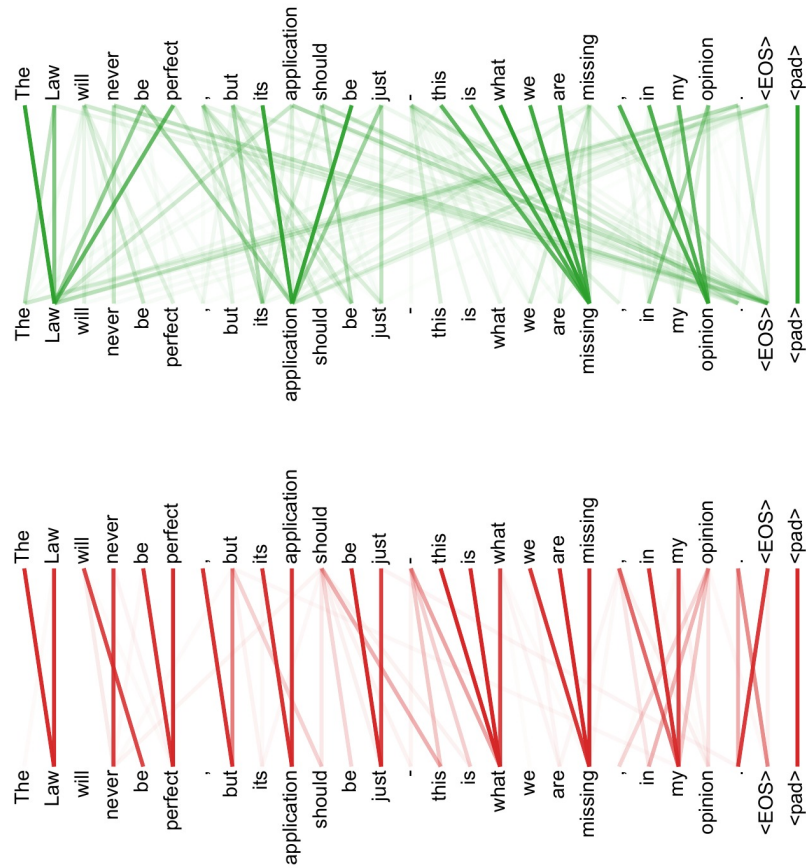


Figure 5: Many of the attention heads exhibit behaviour that seems related to the structure of the sentence. We give two such examples above, from two different heads from the encoder self-attention at layer 5 of 6. The heads clearly learned to perform different tasks.

图 5：许多注意力头呈现出似乎与句子结构相关的行为。上图给出两个例子，它们来自编码器第 5 层（共 6 层）自注意力中的两个不同注意力头。可以清楚看出，不同头学会了执行不同任务。